LAPORAN PRAKTIKUM BASIS DATA

PERTEMUAN 8

MONGODB

Disusun oleh :

Nama	: **Azura Rizky Annisa**
NIM	: 24/535715/SV/24290
Kelas	: B1
Tanggal	: Rabu, 23 April 2025
Dosen Pengampu	: Anni Karimatul Fauziyyah, S.Kom., M.Eng.

PROGRAM STUDI SARJANA TERAPAN

TEKNOLOGI REKAYASA INTERNET

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

SEKOLAH VOKASI

UNIVERSITAS GADJAH MADA

2025

UNIT 8

MONGODB

A. Tujuan Praktikum

Tujuan praktikum ini adalah :

1. Memahami konsep dasar database NoSQL menggunakan MongoDB.
2. Melatih kemampuan melakukan operasi insert, find, dan update pada MongoDB.
3. Mengetahui perbedaan antara insertOne(), insertMany(), updateOne(), updateMany(), dan findOneAndUpdate().
4. Menerapkan penggunaan query dan operator update seperti $set dan parameter multi dalam pembaruan data.

B. Dasar Teori

MongoDB merupakan sistem database NoSQL berbasis dokumen yang menggunakan format BSON (Binary JSON) untuk menyimpan data. Dalam MongoDB, data tidak disimpan dalam tabel seperti pada database relasional, melainkan dalam bentuk dokumen yang dikelompokkan ke dalam koleksi. Beberapa operasi dasar yang sering digunakan adalah insertMany() untuk menambahkan banyak dokumen sekaligus ke dalam koleksi, find() untuk mengambil data berdasarkan kondisi tertentu, serta updateOne(), updateMany(), dan findOneAndUpdate() untuk memperbarui data yang sudah ada. Operator $set digunakan untuk mengubah nilai dari field tertentu dalam dokumen, sedangkan parameter multi:true (pada metode lama) mengizinkan banyak dokumen untuk diubah sekaligus. MongoDB juga secara otomatis memberikan setiap dokumen sebuah _id unik berupa ObjectId. Seiring perkembangan MongoDB, metode update() sudah dianggap deprecated dan dianjurkan menggunakan metode terbaru seperti updateOne(), updateMany(), atau bulkWrite() untuk memastikan kompatibilitas dan kinerja yang lebih baik.

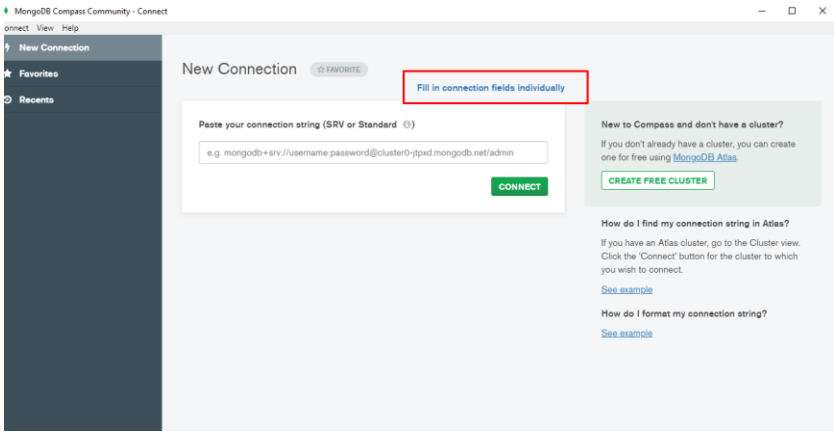
C. Alat dan Bahan

Praktikum ini menggunakan alat dan bahan, yaitu:

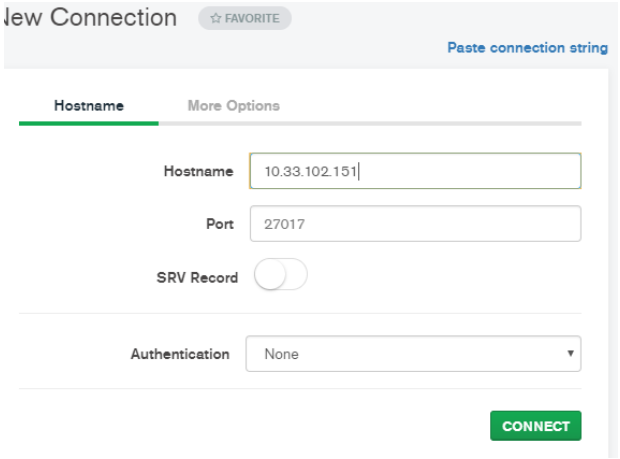
1. Internet
2. Device
3. Software MongoDB Compass

D. Hasil dan Pembahasan

1. Untuk mengakses mongodb dari Mongodb Compass : https://www.mongodb.com/products/compass
   (install terlebih dahulu aplikasi mongodb compass)
   Saat mengakses client mongodb baik dengan console maupun dengan mongodb compass, server mongodb "mongod" harus tetap running



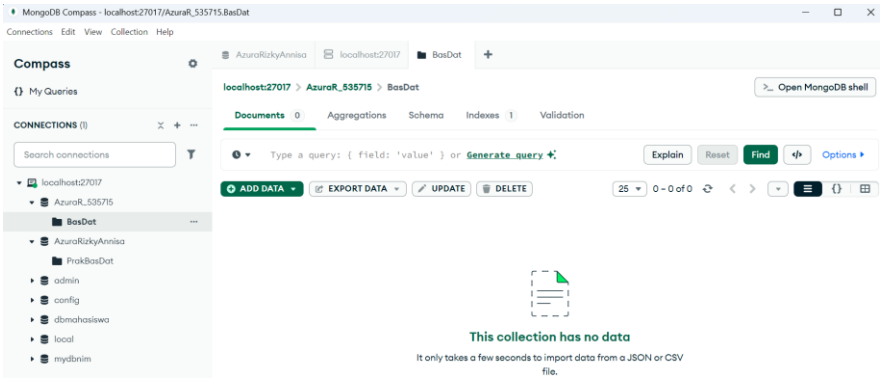
2. Pilih fill in connection



3. Membuat database, collection, dokumen dan insert dokumen di mongodb

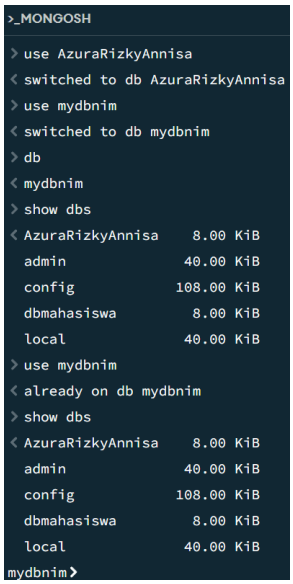
MySQL	MongoDB
Table	Collection
Row	Document
Column	Field
Joins	Embedded documents, linking

- Membuat database baru : buat nama database sertakan nim



Nama database yang dibuat harus mencantumkan NIM, sesuai instruksi tugas. Sebagai contoh, database yang dibuat bernama AzuraR_535715, di mana "AzuraR" adalah nama dan "535715" adalah NIM mahasiswa. Setelah membuat database tersebut, dibuat juga sebuah koleksi (collection) baru di dalamnya bernama Basdat. Koleksi ini nantinya akan digunakan untuk menyimpan berbagai dokumen data. Pembuatan database dan koleksi ini dapat dilakukan dengan menggunakan antarmuka MongoDB Compass, tanpa perlu mengetikkan perintah secara manual, cukup dengan klik tombol "Create Database", lalu mengisi nama database dan nama koleksi yang diinginkan.

- Hapus database



Gambar tersebut menunjukkan penggunaan MongoDB shell untuk mengelola dua database, AzuraRizkyAnnisa dan mydbnim. Dengan perintah use, konteks beralih antara kedua database, yang diikuti oleh perintah show dbs untuk menampilkan semua database beserta ukuran masing-masing. Database AzuraRizkyAnnisa memiliki ukuran 8.00 KiB, sedangkan admin dan local masing-masing berukuran 40.00 KiB, serta config dan dbmahasiswa dengan ukuran 108.00 KiB dan 8.00 KiB. Setelah beralih kembali ke mydbnim, hasil perintah show dbs masih menunjukkan struktur yang sama, menunjukkan akses yang berhasil ke kedua database. Proses ini menggambarkan pentingnya pengelolaan database yang sistematis dan pemahaman tentang potensi dampak perintah-perintah berisiko, seperti db.dropDatabase(), yang dapat menghapus seluruh database beserta isinya secara permanen.

- Membuat collection dapat menggunakan statemen 2 atau menggunakan statemen 4 dengan menambahkan spesifikasi collection dari capped true, index true, dan ukuran collection

```
>_MONGOSH
> use AzuraRizkyAnnisa
< switched to db AzuraRizkyAnnisa
> use mydbnim
< switched to db mydbnim
> db.createCollection("mahasiswa")
< { ok: 1 }
> show collections
< mahasiswa
> db.createCollection("mycol", {capped:true, autoIndexID:true, size:6142800, max: 10000})
✖ ▸ MongoServerError[IDLUnknownField]: BSON field 'create.autoIndexID' is an unknown field.
> db.createCollection("mycol", {capped:true, size:6142800, max: 10000})
< { ok: 1 }
```

Setelah database dibuat, langkah berikutnya adalah membuat collection di dalam database tersebut. Pembuatan collection dapat dilakukan dengan beberapa cara. Cara pertama adalah menggunakan perintah sederhana db.createCollection('mahasiswa'), yang akan membuat collection baru bernama mahasiswa tanpa spesifikasi tambahan. Cara kedua adalah membuat collection dengan spesifikasi tertentu, seperti collection yang bertipe capped. Collection capped memiliki ukuran maksimum tertentu dan akan menimpa data lama jika sudah penuh. Untuk membuat collection capped, digunakan perintah seperti db.createCollection('mycol', { capped: true, size: 6142800, max: 10000 }). Pada percobaan ini, terjadi error karena ada field autoIndexID yang tidak dikenali, sehingga perlu diperbaiki dengan menghapus field tersebut. Setelah dikoreksi, perintah berhasil dijalankan dan collection mycol berhasil dibuat dengan sifat capped, ukuran maksimal 6142800 bytes, dan jumlah maksimal 10000 dokumen.

- Menghapus collection

```
> db.mycol.drop()
< true
> show collections
< mahasiswa
```

Setelah membuat beberapa collection, langkah berikutnya adalah menghapus collection yang tidak diperlukan lagi. Pada contoh ini, collection yang dihapus adalah mycol. Proses penghapusan dilakukan dengan menggunakan perintah db.mycol.drop(). Jika penghapusan berhasil, MongoDB akan memberikan respon true yang menandakan bahwa collection telah dihapus dari database. Untuk memastikan bahwa collection benar-benar sudah terhapus, digunakan perintah show collections. Hasilnya menunjukkan bahwa hanya collection mahasiswa yang tersisa, sedangkan collection mycol sudah tidak ada lagi di dalam database.

- Insert single dokumen



Setelah membuat collection, langkah selanjutnya adalah menambahkan data (insert document) ke dalam collection. Pada contoh ini, data dimasukkan ke dalam collection bernama users menggunakan perintah db.users.insert(). Data yang dimasukkan berbentuk dokumen JSON yang berisi beberapa field, seperti praktikum, description, by, url, dan tags. Field tags sendiri berupa array yang berisi beberapa kata kunci terkait. Setelah perintah dijalankan, MongoDB mengembalikan respon bahwa operasi berhasil dengan menampilkan acknowledged: true dan insertedIds yang menunjukkan ID unik (ObjectId) dari dokumen yang baru saja ditambahkan. Ini menandakan bahwa data sudah tersimpan dengan sukses di dalam collection users.



Pada tahap ini dilakukan proses menambahkan satu dokumen ke dalam collection menggunakan perintah insertOne(). Perintah db.empDetails.insertOne() digunakan untuk memasukkan satu dokumen baru ke dalam collection empDetails. Dokumen yang dimasukkan berisi beberapa field, yaitu First_Name, Last_Name, Date_Of_Birth, e_mail, dan phone, yang masing-masing berisi data pribadi seorang individu. Setelah menjalankan perintah ini, MongoDB memberikan respon acknowledged: true yang berarti operasi penyisipan berhasil, serta menampilkan insertedID yang merupakan ObjectId unik untuk dokumen tersebut. Ini menunjukkan bahwa data karyawan telah berhasil disimpan ke dalam database.

- Insert many dokumen



Pada bagian ini, dilakukan proses menambahkan lebih dari satu dokumen ke dalam sebuah collection sekaligus menggunakan perintah insert(). Dengan perintah db.post.insert([]), beberapa dokumen bisa langsung dimasukkan dalam satu eksekusi. Dokumen-dokumen yang dimasukkan masing-masing berisi field seperti praktikum, description, by, url, dan tags, yang berisi informasi mengenai MongoDB dan NoSQL database. Setelah perintah dijalankan, MongoDB memberikan respon acknowledged: true sebagai tanda bahwa operasi berhasil, dan juga memberikan daftar insertedIds untuk setiap dokumen yang baru saja disisipkan. Setiap dokumen memiliki ObjectId unik masing-masing sebagai identitas di dalam database.

```
> db.empDetails.insertMany([
    {
        First_Name: "Radhika",
        Last_Name: "Sharma",
        Date_Of_Birth: "1995-09-26",
        e_mail: "radhika_sharma.123@gmail.com",
        phone: "9000012345"
    },
    {
        First_Name: "Rachel",
        Last_Name: "Christopher",
        Date_Of_Birth: "1990-02-16",
        e_mail: "Rachel_Christopher.123@gmail.com",
        phone: "9000054321"
    },
    {
        First_Name: "Fathima",
        Last_Name: "Sheik",
        Date_Of_Birth: "1990-02-16",
        e_mail: "Fathima_Sheik.123@gmail.com",
        phone: "9000054321"
    }
])
< {
```

```
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6808558c6159ffae0c9e0a88'),
      '1': ObjectId('6808558c6159ffae0c9e0a89'),
      '2': ObjectId('6808558c6159ffae0c9e0a8a')
    }
  }
mydbnim>
```

Pada tahap ini dilakukan proses penyisipan banyak dokumen sekaligus ke dalam collection menggunakan perintah insertMany(). Perintah db.empDetails.insertMany() digunakan untuk memasukkan beberapa dokumen baru ke dalam collection empDetails dalam satu operasi. Setiap dokumen berisi data karyawan dengan field First_Name, Last_Name, Date_Of_Birth, e_mail, dan phone. Setelah dijalankan, MongoDB memberikan respon acknowledged: true sebagai tanda bahwa seluruh data berhasil disimpan, dan insertedIds menampilkan ObjectId unik untuk masing-masing dokumen yang baru ditambahkan.

- Menggunakan find() untuk menampilkan dokumen pada collection



Pada gambar tersebut digunakan perintah db.post.find() untuk menampilkan seluruh data yang ada di dalam collection post. Perintah ini akan mengembalikan semua dokumen dalam bentuk objek JSON, lengkap dengan field-field yang sebelumnya telah diinsert, seperti praktikum, description, by, url, dan tags. Setiap dokumen juga otomatis memiliki field _id yang merupakan ObjectId unik yang di-generate oleh MongoDB untuk membedakan setiap dokumen. Dengan find(), pengguna dapat melihat isi database secara keseluruhan tanpa memberikan filter atau kondisi pencarian tertentu.



Pada gambar ini digunakan perintah db.post.find().pretty() yang berfungsi untuk menampilkan seluruh dokumen dalam collection post dengan format yang lebih rapi dan mudah dibaca. Fungsi pretty() akan membuat struktur data ditampilkan dengan penataan indentasi yang baik, sehingga setiap field dalam dokumen terlihat lebih teratur. Data yang ditampilkan sama seperti pada perintah find() biasa, hanya saja dengan tampilan yang lebih estetis dan nyaman untuk dianalisis, terutama saat dokumen memiliki banyak field atau struktur yang kompleks.



Pada gambar ini digunakan perintah db.post.findOne({praktikum: "MongoDB"}) yang bertujuan untuk mencari satu dokumen dalam collection post yang memiliki field praktikum bernilai "MongoDB". Perintah findOne() akan mengembalikan hanya satu dokumen pertama yang cocok dengan kriteria pencarian, walaupun ada lebih dari satu dokumen yang memenuhi syarat. Dokumen yang ditampilkan berisi informasi lengkap, termasuk _id, praktikum, description, by, url, dan tags, sehingga hasil pencarian ini sangat berguna untuk menemukan data spesifik dengan cepat.

- Menggunakan kombinasi find dan agregasi sederhana

```
> db.empDetails.insertMany([
    {
        "First_Name": "Radhika",
        "Last_Name": "Sharma",
        "Age": "26",
        "e_mail": "radhika_sharma.123@gmail.com",
        "phone": "9000012345"
    },
    {
        "First_Name": "Rachel",
        "Last_Name": "Christopher",
        "Age": "27",
        "e_mail": "Rachel_Christopher.123@gmail.com",
        "phone": "9000054321"
    },
    {
        "First_Name": "Fathima",
        "Last_Name": "Sheik",
        "Age": "24",
        "e_mail": "Fathima_Sheik.123@gmail.com",
        "phone": "9000054321"
    }
])
```

```
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('680857966159ffae0c9e0a8b'),
      '1': ObjectId('680857966159ffae0c9e0a8c'),
      '2': ObjectId('680857966159ffae0c9e0a8d')
    }
  }
```

Gambar tersebut memperlihatkan penggunaan perintah insertMany() pada koleksi empDetails di MongoDB. Perintah ini digunakan untuk menyisipkan banyak dokumen

sekaligus dalam satu eksekusi. Dalam contoh tersebut, tiga dokumen dimasukkan ke koleksi empDetails, masing-masing berisi informasi karyawan seperti First_Name, Last_Name, Age, e_mail, dan phone. Setelah proses insertMany() dijalankan, MongoDB memberikan respons yang menunjukkan bahwa operasi berhasil (acknowledged: true) dan menampilkan daftar insertedIds yang merupakan ObjectId unik yang otomatis dibuat untuk setiap dokumen baru yang dimasukkan.

- Update dokumen

```
> db.post.update(
      { "praktikum": "MongoDB" },
      { $set: { "praktikum": "New MongoDB Tutorial" } }
  )
< DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
```

Gambar tersebut menunjukkan penggunaan perintah update() pada koleksi post di MongoDB. Query db.post.update({'praktikum': 'MongoDB'}, {$set: {'praktikum': 'New MongoDB Tutorial'}}) digunakan untuk mengubah nilai field praktikum dari 'MongoDB' menjadi 'New MongoDB Tutorial' pada dokumen yang sesuai dengan kondisi. Terlihat adanya peringatan (warning) bahwa metode update() sudah deprecated (tidak disarankan lagi), dan sebaiknya menggunakan updateOne(), updateMany(), atau bulkWrite() untuk praktik terbaru. Hasil eksekusi menunjukkan matchedCount: 1 dan modifiedCount: 1, artinya satu dokumen ditemukan dan berhasil diperbarui

- Update dokumen dengan parameter multi

```
> db.post.update(
      { "praktikum": "MongoDB Tutorial" },
      { $set: { "praktikum": "New MongoDB Tutorial" } },
      { multi: true }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 0,
    modifiedCount: 0,
    upsertedCount: 0
  }
```

Pada gambar tersebut, ditunjukkan penggunaan perintah update() pada koleksi post dengan menambahkan parameter {multi: true}. Query db.post.update({'praktikum':'MongoDB Tutorial'}, {$set:{'praktikum':'New MongoDB Tutorial'}}, {multi:true}) bertujuan untuk memperbarui semua dokumen yang memiliki nilai praktikum sama dengan 'MongoDB Tutorial'. Penambahan opsi multi: true menginstruksikan MongoDB untuk memodifikasi banyak dokumen sekaligus, bukan hanya satu. Namun, hasil eksekusi menunjukkan bahwa matchedCount dan modifiedCount bernilai 0, yang berarti tidak ada dokumen yang cocok dengan kriteria pencarian, sehingga tidak ada perubahan yang terjadi pada koleksi.

- Update dengan findOne

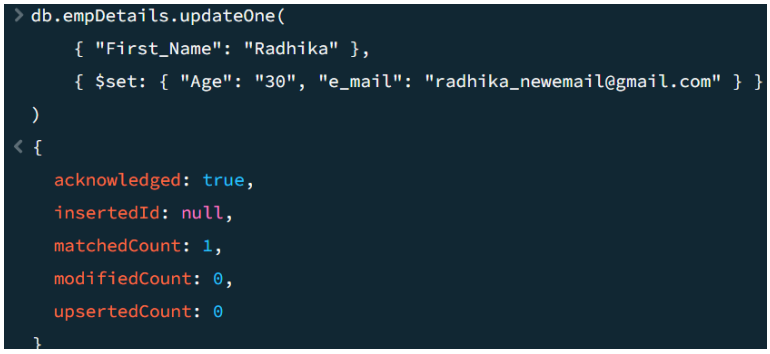
```
>_MONGOSH
> db.empDetails.insertMany([
    {
        First_Name: "Radhika",
        Last_Name: "Sharma",
        Age: "26",
        e_mail: "radhika_sharma.123@gmail.com",
        phone: "9000012345"
    },
    {
        First_Name: "Rachel",
        Last_Name: "Christopher",
        Age: "27",
        e_mail: "Rachel_Christopher.123@gmail.com",
        phone: "9000054321"
    },
    {
        First_Name: "Fathima",
        Last_Name: "Sheik",
        Age: "24",
        e_mail: "Fathima_Sheik.123@gmail.com",
        phone: "9000054321"
    }
])
< {
```

```
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('680859d26159ffae0c9e0a8e'),
    '1': ObjectId('680859d26159ffae0c9e0a8f'),
    '2': ObjectId('680859d26159ffae0c9e0a90')
  }
}
```

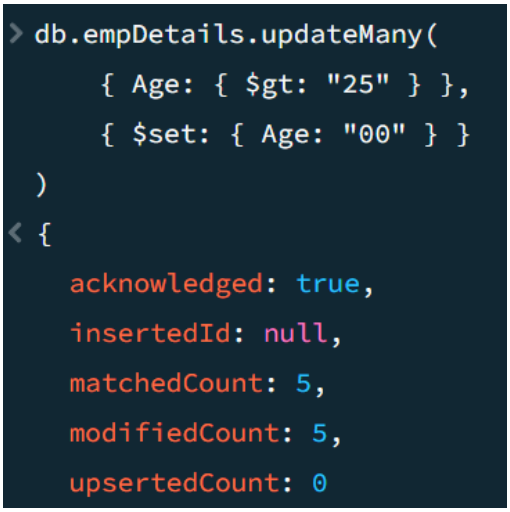
Pada gambar tersebut, terlihat penggunaan perintah insertMany() pada koleksi empDetails untuk menambahkan banyak dokumen sekaligus ke dalam database. Tiga dokumen yang masing-masing berisi informasi seperti First_Name, Last_Name, Age, e_mail, dan phone dimasukkan secara bersamaan. Setelah perintah dieksekusi, MongoDB memberikan respon dengan acknowledged: true, yang berarti permintaan berhasil diproses, serta menampilkan insertedIds yang berisi ObjectId unik untuk setiap dokumen yang baru saja dibuat.



Pada gambar tersebut, digunakan perintah findOneAndUpdate() pada koleksi empDetails. Perintah ini bertujuan untuk mencari satu dokumen yang memiliki First_Name bernama "Radhika", lalu memperbarui nilai Age menjadi 30 dan e_mail menjadi 'radhika_newemail@gmail.com'. Setelah perintah dijalankan, MongoDB mengembalikan data dokumen sebelum dilakukan update, sesuai dengan perilaku default findOneAndUpdate(). Data lama yang ditampilkan memperlihatkan bahwa perubahan belum terlihat di hasil keluaran, namun pada database sebenarnya, field Age dan e_mail sudah diperbarui. Metode ini sangat berguna untuk melakukan update sekaligus mengetahui data sebelumnya.



Pada gambar ini, dilakukan operasi updateOne() pada koleksi empDetails. Perintah tersebut mencari satu dokumen yang memiliki First_Name bernama "Radhika" dan kemudian memperbarui field Age menjadi 30 serta e_mail menjadi 'radhika_newemail@gmail.com'. Hasil dari perintah ini menunjukkan bahwa operasi berhasil (acknowledged: true) dan satu dokumen ditemukan (matchedCount: 1), namun tidak ada perubahan data (modifiedCount: 0) karena kemungkinan nilai Age dan e_mail yang ingin diset sudah sama dengan nilai yang ada saat ini.



Pada gambar ini dilakukan operasi updateMany() terhadap koleksi empDetails. Query ini mencari semua dokumen yang memiliki nilai Age lebih besar dari "25" ($gt: "25"), kemudian mengubah field Age tersebut menjadi "00". Hasil eksekusi menunjukkan bahwa perintah berhasil (acknowledged: true), dengan 5 dokumen yang cocok (matchedCount: 5) dan seluruhnya berhasil diperbarui (modifiedCount: 5). Ini menunjukkan bahwa ada lima data karyawan dengan umur lebih dari 25 tahun yang diupdate serempak menggunakan satu perintah updateMany().

DAFTAR PUSTAKA

MongoDB, Inc. (2024). *MongoDB Manual*. Diakses dari https://www.mongodb.com/docs/manual/

Chodorow, K. (2019). *MongoDB: The Definitive Guide* (3rd ed.). Sebastopol: O'Reilly Media.

Kurniawan, B. (2020). *MongoDB untuk Pemula*. Jakarta: Elex Media Komputindo.